ARTIFICIAL INTELLIGENCE-CSA1717

8 PUZZLE PROBLEM

SLOT-D

REGNO.192512417

Aim:
To implement and study the 8 Puzzle Problem using Python.

Algorithm:
1. Start
2. Initialize input.
3. Apply the algorithm.
4. Display the solution.
5. Stop.

Source Code:

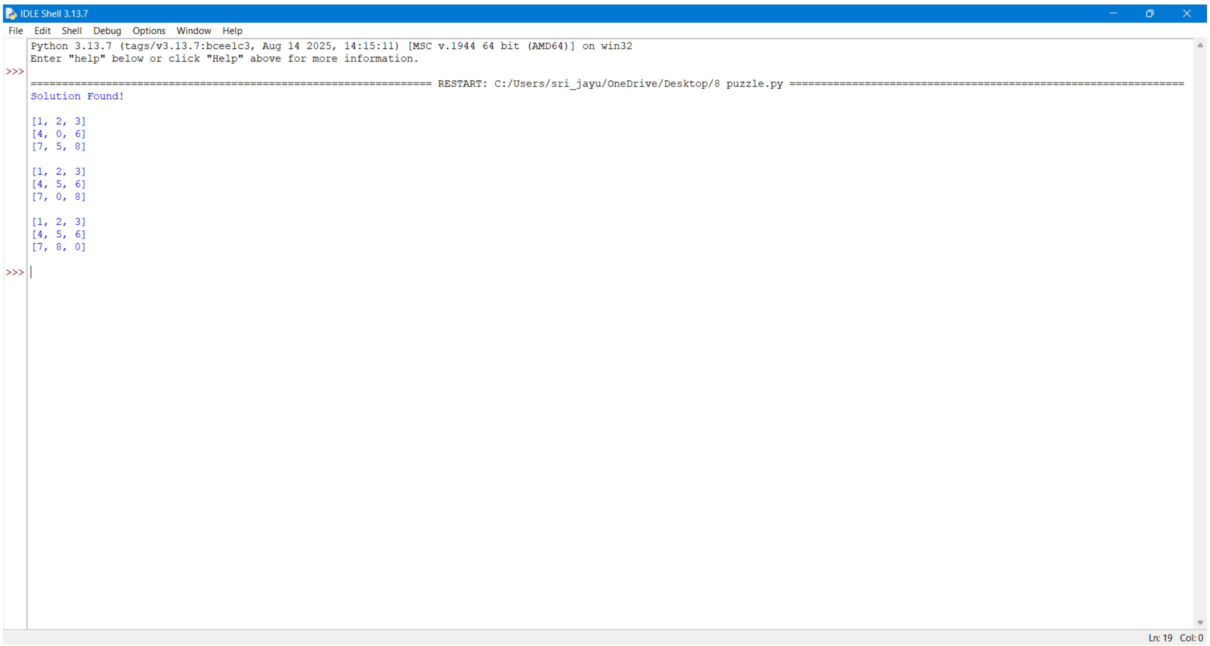
```
from collections import deque


# Goal state

goal = [

    [1, 2, 3],

    [4, 5, 6],

    [7, 8, 0]

]


# Initial state

start = [

    [1, 2, 3],

    [4, 0, 6],

    [7, 5, 8]

]


# Find blank tile (0)
```

```
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate neighboring states
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    directions = [(-1,0),(1,0),(0,-1),(0,1)]

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]

            new_state[x][y], new_state[nx][ny] = \
            new_state[nx][ny], new_state[x][y]

            neighbors.append(new_state)

    return neighbors

# Breadth First Search
def bfs(start, goal):
```

```
    queue = deque()
    queue.append((start, []))

    visited = set()

    while queue:

        state, path = queue.popleft()

        state_tuple = tuple(tuple(row) for row in state)

        if state_tuple in visited:
            continue

        visited.add(state_tuple)

        if state == goal:
            return path + [state]

        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))

    return None

solution = bfs(start, goal)

if solution:
    print("Solution Found!\n")
```

```
    for step in solution:
        for row in step:
            print(row)
        print()

else:
    print("No Solution Exists.")
```

OUTPUT:



Result:

Thus, the 8 Puzzle Problem was implemented successfully.